

Практическое занятие 3: Ветвление, Циклы и ООП

Часть 1: Ветвление

Задание 1.1: if/else — Классификация числа

Напишите класс `NumberClassifier` с методом `static String classify(int number)`, который классифицирует число: "отрицательное" если ≤ 0 ; "ноль" если $= 0$; "однозначное" если от 1 до 9; "двузначное" если от 10 до 99; "трёхзначное" если от 100 до 999; "большое" если ≥ 1000 . Продемонстрируйте работу на числах: -5, 0, 7, 42, 100, 1000, -999.

Ожидаемый вывод:

```
-5 -> отрицательное
0 -> ноль
7 -> однозначное
42 -> двузначное
100 -> трёхзначное
1000 -> большое
-999 -> отрицательное
```

Задание 1.2: switch — Оценка успеваемости

Реализуйте класс `GradeChecker` с двумя вариантами метода `static String getGrade(int score)`: первый — через классический `switch`, второй — через стрелочный `switch` (Java 14+). Диапазоны оценок: 90-100 → "Отлично (A)", 80-89 → "Хорошо (B)", 70-79 → "Удовлетворительно (C)", 60-69 → "Слабо (D)", иначе → "Неудовлетворительно (F)". Протестируйте на значениях 95, 85, 73, 62, 45, 100, 0.

Задание 1.3: switch с паттерн-матчингом (Java 17+)

Напишите метод `static String describe(Object obj)` с использованием `switch` с паттерн-матчингом (Java 17+). Обрабатываемые случаи: `null`, `Integer i` (с пометкой положительной/не положительной), `String s` когда пустая, `String s` непустая, `Double d`, `int[] arr`, иные объекты (через `default`). Протестируйте на: `null`, 42, -5, "", "Привет", 3.14, `new int[]{1, 2, 3}`, `true`.

Часть 2: Циклы

Задание 2.1: Таблица умножения

Напишите программу, которая выводит таблицу умножения от 1 до 10 в форматированном виде (числа выровнены по столбцам шириной 4). Используйте вложенные `for`-циклы. Каждая строка начинается с номера строки, затем все произведения.

Задание 2.2: Числа Фибоначчи

Реализуйте класс `Fibonacci` с методами: `fibIterative(int n)` — итеративное вычисление с `while`-циклом; `fibFor(int n)` — с `for`-циклом. Выведите F(0)..F(15). Найдите первое число Фибоначчи, превышающее 1000.

Задание 2.3: Работа со строками в цикле

Реализуйте класс `StringProcessor` со статическими методами: `countVowels(String text)` — подсчёт гласных букв (русских и английских); `isPalindrome(String text)` — проверка на палиндром без учёта регистра и знаков препинания (используйте сравнение символов с двух концов строки); `reverse(String text)` — реверс строки без `StringBuilder` (с двумя указателями); `findLongestWord(String sentence)` — самое длинное слово в предложении. Протестируйте на: "Привет, Java-разработчик!", "ronott", "Madañ", "hello", "A rosa uana na lamy Asopa", "The quick brown fox jumps over the lazy dog".

Задание 2.4: break, continue и метки

Изучите и запустите программу. Объясните: (1) как работают метки `outer:` и `search:`; (2) что делает `continue outer` на строке с проверкой делителей; (3) какой результат выведет каждый из трёх блоков программы?

```
public class LoopControl {
    public static void main(String[] args) {
        // Блок 1: Найти простое простое число больше 100
        System.out.println("--- Первое простое > 100 ---");
        int n = 101;
        outer:
        while (true) {
            if (n % 2 == 0 && n != 2) {
                n++;
                continue;
            }
            for (int d = 3; d * d <= n; d += 2) {
                if (n % d == 0) {
                    n++;
                    continue outer; // Перейти к следующей итерации while
                }
            }
            break; // n — простое
        }
        System.out.println("Первое простое > 100: " + n);

        // Блок 2: Распечатать только нечётные числа, пропуская кратные 3
        System.out.println("\n--- Нечётные, не кратные 3 (от 1 до 20) ---");
        for (int i = 1; i <= 20; i++) {
            if (i % 2 == 0) continue; // Пропустить чётные
            if (i % 3 == 0) continue; // Пропустить кратные 3
            System.out.print(i + " ");
        }
        System.out.println();

        // Блок 3: Поиск в матрице — найти и сразу выйти из обоих циклов
        System.out.println("\n--- Поиск в матрице ---");
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int target = 5;
        int foundRow = -1, foundCol = -1;

        search:
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                if (matrix[row][col] == target) {
                    foundRow = row;
                    foundCol = col;
                    break search; // Выйти из ОБОИХ циклов
                }
            }
        }

        if (foundRow != -1) {
            System.out.printf("Число %d найдено на позиции [%d][%d]\n", target, foundRow, foundCol);
        }
    }
}
```

Часть 3: Наследование и инкапсуляция

Задание 3.1: Иерархия транспортных средств

Проектируйте и реализуйте иерархию классов для автопарка:

- Абстрактный класс `Vehicle`: поля `brand`, `model`, `year` (private), `fuelLevel` (0.0-1.0); геттеры для всех полей; сеттер для `fuelLevel` с проверкой диапазона; абстрактные методы `getFuelConsumption()` (л/100км) и `getType()`; конкретные методы `calculateFuelNeeded(double distance)` и `canTravel(double distance, double tankCapacityLiters)`; переопределённый `toString()`.
- Класс `Car` extends `Vehicle`: поля `doors`, `automatic`; `getFuelConsumption()` возвращает 9.5 (авт.) или 8.0 (мех.); метод `honk()`.
- Класс `Truck` extends `Vehicle`: поле `cargoCapacityTons`; `getFuelConsumption()` = 20 + `cargoCapacity` × 3.
- Интерфейс `Electric`: методы `getBatteryLevel()`, `getRangeKm()`, `charge(double hours)`.
- Класс `ElectricCar` extends `Car` implements `Electric`: поля `batteryLevel`, `maxRangeKm`; `charge` увеличивает батарею на 20%/час (макс. 100%); `getFuelConsumption()` возвращает 0.

Создайте `List<Vehicle>` с четырьмя машинами (Toyota Camry, Lada Vesta, Kamaaz, Tesla Model 3) и выведите для каждой: тип, расход топлива на 500 км, и для электромобилей — запас хода. Продемонстрируйте полиморфизм: для `Car` вызовите `honk()`, для `Electric` — `getRangeKm()`.

Задание 3.2: Инкапсуляция — Банковский счёт

Реализуйте класс `BankAccount` с полями (все private): `accountNumber` (final String), `balance` (double), `owner` (final String), `failedAttempts` (int), `blocked` (boolean), `pin` (String — нет публичного геттера). Методы: `withdraw(String enteredPin, double amount)` — если счёт заблокирован, отказ; при неверном PIN увеличивает `failedAttempts`, при 3 неверных блокирует счёт; при верном PIN сбрасывает `failedAttempts`, проверяет сумму и вычитает; `deposit(double amount)` — проверяет `amount > 0`; `validatePin(String pin)`, `getMaskedBalance()` — скрывает суммы свыше 100 000. Напишите `toString()` с пометкой `[ЗАБЛОКИРОВАН]` если счёт заблокирован.

Часть 4: Полиморфизм

Задание 4.1: Система скидок (перегрузка методов)

```
public class DiscountCalculator {

    // Перегрузка метода calculateDiscount:
    // 1. По типу клиента (String)
    public static double calculateDiscount(double price, String customerType) {
        return switch (customerType.toLowerCase()) {
            case "vip" -> price * 0.30;
            case "regular" -> price * 0.10;
            case "new" -> price * 0.05;
            default -> 0;
        };
    }

    // 2. По количеству покупок
    public static double calculateDiscount(double price, int purchaseCount) {
        if (purchaseCount >= 100) return price * 0.20;
        if (purchaseCount >= 50) return price * 0.15;
        if (purchaseCount >= 10) return price * 0.10;
        return 0;
    }

    // 3. По промокоду
    public static double calculateDiscount(double price, String promoCode, boolean isFirstOrder) {
        double discount = 0;
        if ("SAVE10".equals(promoCode)) discount = price * 0.10;
        if ("SAVE20".equals(promoCode)) discount = price * 0.20;
        if (isFirstOrder) discount += price * 0.05; // Дополнительная скидка новым
        return Math.min(discount, price * 0.50); // Не более 50%
    }

    public static void main(String[] args) {
        double price = 10000.0;

        System.out.println("Скидка VIP-клиента: " + calculateDiscount(price, "vip") + " руб.");
        System.out.println("Скидка за 75 покупок: " + calculateDiscount(price, 75) + " руб.");
        System.out.out.println("Скидка SAVE20 с первой заказ: " +
            calculateDiscount(price, "SAVE20", true) + " руб.");
    }
}
```

Дополните класс четвёртой перегрузкой `calculateDiscount` (например, по возрасту клиента, сезону или категории товара). Запустите программу и убедитесь, что все перегрузки работают корректно.

Задание 4.2: Геометрические фигуры (переопределение методов)

Реализуйте классы `Circle` и `Rectangle`. Реализуйте `Square extends Rectangle`: конструктор принимает одну сторону `side` и передаёт её как оба параметра в конструктор `Rectangle`; переопределите `draw()`, чтобы выводилось слово «квадрат» вместо «прямоугольник». Запустите `ShapeTest` с добавленным `Square`.

```
public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public abstract double area();
    public abstract double perimeter();
    public abstract void draw(); // Симуляция рисования

    // Метод сравнения по площади
    public int compareArea(Shape other) {
        return Double.compare(this.area(), other.area());
    }

    @Override
    public String toString() {
        return String.format("%s[цвет=%s, S=%.2f, P=%.2f]",
            getClass().getSimpleName(), color, area(), perimeter());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        if (radius <= 0) throw new IllegalArgumentException("Радиус должен быть > 0");
        this.radius = radius;
    }

    @Override
    public double area() { return Math.PI * radius * radius; }

    @Override
    public double perimeter() { return 2 * Math.PI * radius; }

    @Override
    public void draw() {
        System.out.println("Рисуем " + color + " круг с радиусом " + radius);
    }
}

public class Rectangle extends Shape {
    protected double width;
    protected double height;

    public Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() { return width * height; }

    @Override
    public double perimeter() { return 2 * (width + height); }

    @Override
    public void draw() {
        System.out.printf("Рисуем %s прямоугольник %.1f x %.1f\n", color, width, height);
    }
}

public class ShapeTest {
    public static void main(String[] args) {
        List<Shape> shapes = new ArrayList<>();
        shapes.add(new Circle("красный", 5));
        shapes.add(new Rectangle("синий", 4, 6));
        shapes.add(new Square("зелёный", 5));
        shapes.add(new Circle("жёлтый", 3));

        System.out.println("=== Все фигуры ===");
        for (Shape s : shapes) {
            s.draw();
            System.out.println(" " + s);
        }

        System.out.println("\n=== Сортировка по площади ===");
        shapes.sort(Shape::compareArea);
        shapes.forEach(System.out::println);

        System.out.println("\n=== Только круги ===");
        for (Shape s : shapes) {
            if (s instanceof Circle c) {
                System.out.printf("Круг с радиусом: проверьте через toString: %s\n", c);
            }
        }

        double totalArea = shapes.stream().mapToDouble(Shape::area).sum();
        System.out.printf("Общая площадь: %.2f\n", totalArea);
    }
}
```

Часть 5: Комплексное задание

Задание 5.1: Система управления зоопарком

Реализуйте мини-систему управления зоопарком, применяя все концепции лекции:

- Абстрактный класс `Animal`: поля `name`, `age`, `weight`, `energyLevel` (0-100); методы `eat(int calories)`, `sleep(int hours)`, `makeSound()`.
- Подкласс `Predator`: метод `hunt()` — повышает `energyLevel` на 30, снижает вес на 0.5.
- Подкласс `Herbivore`: метод `graze()` — повышает `energyLevel` на 15.
- `Lion` extends `Predator`: `makeSound()` → "P-p-p!", метод `roar()`.
- `Elephant` extends `Herbivore`: `makeSound()` → "Tyyt!", метод `trumpet()`.
- Интерфейс `Trainable`: `train(String command)`, `listCommands()`. `Lion` реализует `Trainable`.
- Класс `Zoo`: `addAnimal(Animal a)`, `feedAll()`, `makeNoise()`, `getHungryAnimals()` (energyLevel < 30), `findAnimal(String name)` (возвращает `Optional<Animal>`).

Продемонстрируйте работу системы в `main()`.

Часть 6: Контрольные вопросы

Отвечать письменно:

- В каком порядке выполняются конструкторы при создании объекта подкласса?
- Можно ли переопределить `static` метод? Что такое "скрытие метода" (method hiding)?
- Что произойдёт, если `switch` не имеет `default` и ни один `case` не сошёлся?
- Почему в `for-each` нельзя удалять элементы коллекции? Как правильно это делать?
- Чем отличается `break` с меткой от обычного `break`?
- Что означает `@Override`? Что произойдёт, если метод с этой аннотацией не переопределяет родительский?
- Какой модификатор доступа использовать для поля, которое должно быть видно в подклассе, но не снаружи пакета?
- В чём разница между `final` классом и `sealed` классом?
- Можно ли в Java иметь несколько конструкторов? Как они различаются?
- Что такое "ковариантный тип возвращаемого значения" при переопределении?

Результаты занятия

К концу занятия вы должны сдать:

- Файлы `.java` для всех заданий (скомпилированные, без ошибок)
- Ответы на контрольные вопросы

Критерии оценки:

- Корректное использование `@Override`
- Правильная инкапсуляция (приватные поля, геттеры/сеттеры)
- Осысленная иерархия наследования
- Полноморфное поведение через переопределение
- Использование `super`: там, где нужно